

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

***CO4:** Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 10%

QUESTIONS: 5

Duration :60 Mins

Total Marks:5

Assessment Mode: In Class

Annexure A

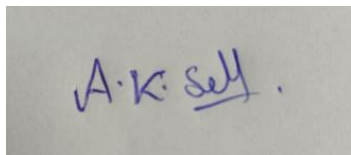
CONCEPT MAPPING

Code of Conduct:

I A K Selva Prabhu(192421224) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A photograph of a handwritten signature in blue ink on a light-colored surface. The signature reads "A.K. Selva" with a period at the end.

33. Intermediate-Code Representation of Parameter Passing

1. What is Parameter Passing?

- Parameter passing is the mechanism by which arguments are passed to a function or procedure.
- It connects actual parameters with formal parameters.
- Intermediate code must represent how these parameters are passed.

2. Why Intermediate-Code Representation?

- Provides a machine-independent representation.
- Helps the compiler understand how arguments are transferred.
- Supports optimization and code generation.
- Preserves the semantics of parameter passing.

3. Parameter Passing Methods

a) Pass by Value

- A copy of the actual parameter is passed.
- Changes made inside the function do not affect the original variable.

TAC example:

t1 = x

param t1

call fun, 1

b) Pass by Reference

- The address of the actual parameter is passed.
- Changes inside the function affect the original variable.

TAC example:

t1 = &x

param t1

call fun, 1

c) Pass by Result (Out)

- The formal parameter receives a value from the procedure.
- The final value is copied back to the actual parameter.

TAC example:

call fun, 1

x = t1

d) Pass by Value-Result (Copy-in/Copy-out)

- Value is copied into the formal parameter initially.
- Modified value is copied back after the procedure finishes.

4. Intermediate-Code Representations

- Three-address code (TAC)
- Parameter instructions
- param
- call
- Address operators such as &
- Temporary variables

5. Example

Source:

```
int x = 10;
```

```
fun(x);
```

Possible intermediate code:

```
t1 = x
```

param t1

call fun, 1

6. Storage and Access

- Parameters may be stored in activation records.
- Access can be through registers, stack locations or memory.
- Intermediate code represents the required parameter information.

7. Important Points

- Parameter passing connects caller and called procedure.
- Different passing methods have different effects.
- TAC provides a machine-independent representation.
- Correct representation is necessary for proper procedure execution.

8. Summary

Parameter passing → Actual parameters → Formal parameters → TAC representation
→ Procedure call

34. Activation Records and Runtime Management of Procedure Calls

1. What is an Activation Record?

An activation record is a block of memory created for each procedure or function call.

It contains information required to execute the procedure and return control to the caller.

2. Components of an Activation Record

An activation record generally contains:

- Return value – stores the value returned by the procedure.
- Return address – address where execution should continue.
- Control link – points to the caller's activation record.
- Access link – provides access to non-local variables when required.

- Actual parameters – arguments passed to the procedure.
- Local variables – variables declared inside the procedure.
- Temporaries – temporary values generated during execution.
- Saved registers – registers that need to be preserved.

3. Run-Time Stack Organization

Activation records are generally maintained using a run-time stack.

Example:

Top of Stack

Activation Record of C

Activation Record of B

Activation Record of A

When a procedure is called:

- A new activation record is pushed.

When it returns:

- Its activation record is popped.

4. Runtime Management of Procedure Calls

Call sequence:

1. Evaluate actual parameters.
2. Store parameters.
3. Save the return address.

4. Create activation record.
5. Transfer control to the called procedure.

Return sequence:

1. Store the return value.
2. Restore saved registers.
3. Remove the activation record.
4. Restore caller's environment.
5. Transfer control back to the caller.

5. Example

For:

```
main() {  
    fun();  
}
```

During fun():

main activation record

↓

fun activation record

After fun() returns:

main activation record

6. Importance

- Supports procedure calls and returns.
- Provides storage for local variables.
- Manages parameters and return values.
- Supports recursion.

- Helps manage run-time memory efficiently.

7. Summary

Procedure call → Create activation record → Execute procedure → Return → Remove activation record

35. Generation of Intermediate Code for Function Invocations

1. What is Function Invocation?

Function invocation means calling a function to execute its operations.

Example:

```
result = add(a, b);
```

The compiler generates intermediate code to:

- Evaluate arguments.
- Pass arguments.
- Call the function.
- Receive the return value.
- Store the result.

2. Steps in Generating Intermediate Code

1. Evaluate each argument.
2. Compute the value of each argument.
3. Place arguments in the appropriate location.
4. Generate parameter instructions.
5. Generate the function call.
6. Store the returned value in a temporary variable.
7. Continue execution after the call.

4. Intermediate-Code Patterns

Call with arguments:

param a

param b

call add, 2

Call with return value:

param a

param b

t1 = call add, 2

Here t1 stores the returned value.

5. Example

Source code:

x = add(a, b);

Possible TAC:

param a

param b

t1 = call add, 2

x = t1

6. Function Call in Three-Address Code

Common TAC instructions:

param x

param y

call fun, 2

t1 = call fun, 2

- param → passes an argument.
- call → invokes a function.
- t1 → stores returned value.

7. Runtime Aspects Reflected in Code

- Arguments are prepared before the call.
- Return address is saved.
- Activation record is created.
- Control transfers to the function.
- Return value is received after execution.

8. Important Points

- Function invocation connects caller and callee.
- Arguments must be evaluated before calling.
- Intermediate code represents parameter passing and return values.
- Temporary variables may store returned values.

9. Summary

Evaluate arguments → Pass parameters → Call function → Execute → Return value → Continue execution

36. Intermediate-Code Representation of Return Operations

1. What is a Return Operation?

A return operation:

- Ends execution of a function/procedure.
- Transfers control back to the caller.

- May return a value to the caller.

Example:

```
return x;
```

2. Why Intermediate-Code Representation?

Return operations need to represent:

- The return value.
- The return address.
- Transfer of control.
- Restoration of the caller's environment.

3. Components Involved

- Return value – value produced by the function.
- Return address – location to continue execution.
- Activation record – contains call-related information.
- Control transfer – transfers control to caller.
- Restoration – restores the caller's environment.

4. Intermediate-Code Forms

a) Return without value

```
return
```

Used by procedures/functions that do not return a value.

b) Return with value

```
return x
```

Returns the value of x.

c) Return with expression

```
t1 = a + b
```

```
return t1
```

The expression is evaluated first and then returned.

5. Typical Intermediate-Code Sequence

For:

```
int add(int a, int b) {  
    return a + b;  
}
```

Possible TAC:

```
t1 = a + b
```

```
return t1
```

Caller:

```
param a
```

```
param b
```

```
t2 = call add, 2
```

6. Activation Record Unwinding

When returning:

1. Compute the return value.
2. Store the return value.
3. Restore saved registers/environment.
4. Remove the current activation record.
5. Transfer control to the caller.

7. Return in Different Situations

Multiple return statements

```
if x > 0
```

```
return x
```

```
else
```

```
return 0
```

Each return transfers control back to the caller.

Return with expression

```
t1 = x * y
```

```
return t1
```

8. Important Points

- Return terminates the current procedure execution.
- It transfers control back to the caller.
- Return values can be stored in temporary locations/registers.
- Activation-record information must be restored.

9. Summary

Evaluate return value → Store result → Restore environment → Transfer control →
Caller continues

37. Intermediate Code Generation for Nested Procedure Invocations

1. What are Nested Procedure Invocations?

A nested procedure invocation occurs when:

- One procedure calls another procedure.
- A procedure call appears inside another procedure call.
- Function calls are nested within expressions.

Example:

```
f(g(x));
```

Here:

- $g(x)$ is called first.
- Its result is passed to $f()$.

2. Why Special Handling is Needed?

Nested calls require the compiler to manage:

- Evaluation order.
- Parameters.
- Return values.
- Multiple activation records.
- Control flow between procedures.

3. General Strategy for Nested Calls

1. Evaluate the innermost call first.
2. Store its return value.
3. Use that value as an argument to the outer call.
4. Generate parameter instructions.
5. Call the outer procedure.
6. Store its return value if required.

4. Intermediate-Code Pattern

For:

$f(g(x));$

Possible TAC:

param x

t1 = call g, 1

param t1

$t2 = \text{call } f, 1$

Here:

- $g(x)$ is evaluated first.
- Result is stored in $t1$.
- $t1$ becomes the argument for f .
- Result of f is stored in $t2$.

5. Multiple Nested Calls

Source:

$f(g(x), h(y));$

Possible TAC:

param x

$t1 = \text{call } g, 1$

param y

$t2 = \text{call } h, 1$

param $t1$

param $t2$

$t3 = \text{call } f, 2$

6. Nested Call with Expression

For:

$f(x + g(y));$

TAC:

param y

t1 = call g, 1

t2 = x + t1

param t2

t3 = call f, 1

7. Activation Records and Run-Time Aspects

For nested calls:

f(g(x))

the run-time stack may contain:

Activation Record of f

↓

Activation Record of g

↓

Current execution

The inner call is completed before the outer call continues.

8. Important Points

- Nested calls are evaluated according to the required evaluation order.
- The result of an inner call can become an argument of an outer call.
- Temporary variables store intermediate results.
- Multiple activation records may exist on the run-time stack.
- Intermediate code must remain machine-independent.

9. Common Three-Address Instructions

param x

t1 = call g, 1

param t1

t2 = call f, 1

Common instructions include:

param

call

t = call

return

10. Summary

Evaluate inner call → Store result → Pass result → Invoke outer call → Return result